Practical pipeline documentation:
LaTeX → Lean4 → Julia for FSA v1.5

Ajay Talati + Claude

2026-05-09

Abstract

This is the evidence-based, file-and-line-cited companion to the *LEAN4-First Charter* (`lean4_first_charter.pdf` in this directory). The charter is the policy doc; this is the implementation doc. Sections 1–4 reproduce the FSA v1.5 mathematics, SMC^2-MPC pipeline, study CLI, and functional-Julia rationale exactly as written in the source documents on disk so this PDF is self-contained and verifiable in isolation. Sections 5–8 are new: they document the build, the JSON dispatch protocol, the Lean4 ↔ Julia function map, the differential test harness, and the verification gap on the GPU files `gpu_pf.jl` and `gpu_control.jl`. Every claim is tagged with a source file path and line range so a future reader (human or agent) can trace the data and information flow without guessing.

Contents

1 The FSA v1.5 model

1.1 State variables

FSA (Fitness–Stress–Autonomic) is a 3-state stochastic differential equation model for an athlete's response to training. State $y = (B, F, A)$:

- B — aerobic fitness (Banister chronic; bounded $[0, 1]$, Jacobi-style state-dependent diffusion).
- F — unified fatigue (Banister acute; ≥ 0, CIR-style diffusion).
- A — autonomic amplitude (Stuart–Landau; ≥ 0, CIR-style diffusion).

1.2 Drift

In `/day` units:

$$\mu(B, F) = \mu_0 + \mu_B B - \mu_F F - \mu_{FF} F^2 \tag{1}$$

$$dB/dt = \kappa_B \left(1 + \varepsilon_A A\right) \Phi(t) - B/\tau_B \tag{2}$$

$$dF/dt = \kappa_F \, \Phi(t) - (1 + \lambda_A A)/\tau_F \, F \tag{3}$$

$$dA/dt = \mu(B, F) \cdot A - \eta A^3 \tag{4}$$

The single exogenous control is $\Phi(t)$, the training-strain rate (≥ 0).

1.3 State-dependent diffusion (Itô)

$$\sigma_B(B)\, dW_B = \sigma_B \sqrt{B(1-B)}\, dW_B \qquad \text{(Jacobi, } [0,1]\text{)} \tag{5}$$

$$\sigma_F(F)\, dW_F = \sigma_F \sqrt{F}\, dW_F \qquad \text{(CIR, } \geq 0\text{)} \tag{6}$$

$$\sigma_A(A)\, dW_A = \sigma_A \sqrt{A}\, dW_A \qquad \text{(CIR, } \geq 0\text{)} \tag{7}$$

The diffusion vanishes at each domain boundary so the SDE keeps the state in its physiological range without clipping.

1.4 Observation channels

Three direct-Gaussian observation channels:

$$y_B^{\text{obs}} = B + \mathcal{N}(0, \sigma^2_{B,\text{obs}}), \qquad y_F^{\text{obs}} = F + \mathcal{N}(0, \sigma^2_{F,\text{obs}}), \qquad y_A^{\text{obs}} = A + \mathcal{N}(0, \sigma^2_{A,\text{obs}}). \tag{8}$$

No HR / sleep / stress / steps channels (that is the v2 surface). No circadian forcing. All three obs noise scales pinned at 0.005.

1.5 Time grid

60 minutes per bin (24 bins per day); $\Phi(t)$ is constant within a bin. The closed-loop bench's stride is 12 bins (12 hours) and its filter window is 24 bins (1 day).

1.6 Parameters: 14 dynamics + 3 obs noise; 10 estimated, 7 pinned

The 14 dynamics parameters use the v1.5 basis: κ_B is replaced by $B_\infty = \kappa_B \tau_B$ and κ_F by $F_\infty = \kappa_F \tau_F$ (steady-state fitness / fatigue at $\Phi = 1$, no A-coupling) for better identifiability under the direct-Gaussian observation model. The basis-rotation adapter is at `simulation.params_v15_to_v1_nt` (Julia) / `simulation.params_v15_to_v1` (Python+JAX).

Symbol	Role	Truth value	Pinned	Prior (estimated only)
τ_B	chronic-B time constant	42.0	yes	—
τ_F	acute-F time constant	7.0	no	LogNormal($\log 7.0,\ 0.30$)
B_∞	steady-state B at $\Phi = 1$	0.504	no	LogNormal($\log 0.504,\ 0.30$)
F_∞	steady-state F at $\Phi = 1$	0.21	no	LogNormal($\log 0.21,\ 0.30$)
ε_A	A-on-B multiplicative coupling	0.40	yes	—
λ_A	A-on-F recovery coupling	1.00	no	LogNormal($\log 1.0,\ 0.30$)
μ_0	Stuart–Landau bias	0.02	no	LogNormal($\log 0.02,\ 0.30$)
μ_B	SL slope on B	0.30	no	LogNormal($\log 0.30,\ 0.30$)
μ_F	SL slope on F	0.10	no	LogNormal($\log 0.10,\ 0.30$)
μ_{FF}	SL curvature on F^2	0.40	yes	—
η	SL cubic damping	0.20	yes	—
σ_B	dynamics noise scale on B	0.010	no	LogNormal($\log 0.010,\ 0.30$)
σ_F	dynamics noise scale on F	0.012	no	LogNormal($\log 0.012,\ 0.30$)
σ_A	dynamics noise scale on A	0.020	no	LogNormal($\log 0.020,\ 0.30$)
$\sigma_{B,\text{obs}}$	obs noise on B channel	0.005	yes	—
$\sigma_{F,\text{obs}}$	obs noise on F channel	0.005	yes	—
$\sigma_{A,\text{obs}}$	obs noise on A channel	0.005	yes	—

Table 1: All 17 v1.5 parameters. 10 are estimated by the filter (LogNormal priors centred on log(truth) with scale 0.30); 7 are pinned at truth (per the FIM-rank decision: option B + pin $\{\tau_B, \eta, \varepsilon_A, \mu_{FF}\}$, plus the 3 obs-noise scales). Truth values read from `T28d_seed42/manifest.json` fields `truth_params` and `pinned_dynamics`; the prior specification is at `version_1_5_Julia/models/fsa_high_res/estimation.jl`, `PARAM_PRIOR_CONFIG`.

1.7 Initial state

Canonical $y_0 = (B_0, F_0, A_0) = (0.05, 0.30, 0.10)$ for the plant. For the closed-loop replan path, the controller is fed the *filter-derived* smoothed end-of-window state $\hat{y}$ (not the plant's true state) so the controller is operating on the same information the filter has.

2 SMC^2-MPC pipeline and cost functional

2.1 The three-stage pipeline

1. **Outer SMC^2 filter.** Tempered sequential Monte Carlo over the 10-dim unconstrained parameter cloud θ. At each tempering level $\lambda \in [0, 1]$ the targets graduate from prior to posterior. Resample (systematic) → Hamiltonian Monte Carlo (HMC) moves with finite-difference gradients → next λ. Returns a posterior cloud over θ for each rolling filter window.

2. **Inner particle filter (PF).** Per θ candidate, an inner PF computes $\log p(y_{1:T} \mid \theta)$ by propagating a state-particle cloud forward through the SDE and reweighting by the observation likelihood. This is the marginal log-likelihood the outer SMC^2 uses.

3. **Forward-simulation controller (NOT a particle filter).** The controller takes the filter-derived posterior mean parameters $\hat{\theta}$ and the filter-derived smoothed end-of-window state $\hat{y}$. It runs an SMC^2 over a low-dimensional schedule parameterisation $\theta_{\text{ctrl}} \in \mathbb{R}^{n_{\text{anchors}}}$ (here $n_{\text{anchors}} = 12$): for each candidate θ_{ctrl}, decode the 12 RBF anchors into a per-bin Φ schedule, simulate the SDE forward $n_{\text{inner}} = 64$ times under common-random-numbers (CRN) noise, accumulate the cost

$$J(\Phi) = -\int_0^{T_{\text{plan}}} A(t)\, dt + \lambda_F \int_0^{T_{\text{plan}}} \left(F(t) - F_{\max}\right)_+^2 dt \tag{9}$$

with $F_{\max} = 0.40$ and $\lambda_F = 1.0$, and use the negative cost as the tempering target. Output is the posterior-mean $\theta^{\star}_{\text{ctrl}}$, decoded to a Φ-schedule for the plant's next stride.

Critical: the controller does not run a particle filter internally. It runs deterministic forward simulations (n_{inner} noise trials per θ_{ctrl}) using $\hat{\theta}$ and $\hat{y}$.

2.2 Schedule parameterisation

The per-bin Φ schedule is a sigmoid of an RBF combination:

$$\Phi(t) = \Phi_{\max} \cdot \text{sigmoid}\left(c_\Phi + \sum_{a=1}^{n_{\text{anchors}}} \theta_{\text{ctrl},a} \cdot \text{rbf}_a(t) \right) \tag{10}$$

with $\Phi_{\max} = 3.0$, $\Phi_{\text{default}} = 1.0$, and $c_\Phi = \text{logit}(\Phi_{\text{default}}/\Phi_{\max})$ so that $\theta_{\text{ctrl}} = 0$ produces $\Phi \equiv \Phi_{\text{default}} = 1.0$ (the canonical Banister training rate prior).

2.3 Per-tempering-level kernel

ChEES (Cascading Hamiltonian-augmented Equi-Energy Sampler) with adaptive leapfrog count picked from the candidate list $[16, 32, 64, \ldots, n_{\text{ChEES,max}}]$ at $n_{\text{ChEES,max}} = 512$, and $n_{\text{HMC}} = 16$ moves per level.

2.4 Replan cadence and rolling windows

Filter strides are 12 bins (12 hours) wide; each new stride incorporates 12 new bins of observation. Replans fire every $K_{\text{replan}} = 2$ strides, so once per simulated day. At each replan the controller plans the *full* T-day horizon (not the remaining horizon); when applied to the plant only the next stride's Φ is consumed before the next replan computes a fresh T-day-ahead schedule.

3 Study configuration and CLI

3.1 Hardware and software

- GPU: NVIDIA GeForce RTX 5090 (read at run time from `manifest.json::device`).
- Framework: `julia/SMC2FC_functional/` (the default; the previous `julia/SMC2FC/` has been renamed to `julia/SMC2FC_DEPRECIATED_USE_FUNCTIONAL/` and is not used by this study). Per the technical guide [2], numerics on the GPU bench are bit-identical to the deprecated framework at the saturated config.

3.2 Minimal CLI

The bench's defaults are the saturated-tier settings; the only flag that varies between horizons is `-T-days`. The minimal invocation per horizon (used by the launcher in §3.6):

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
    --T-days <N> --seed 42 --output-dir <DIR>
```

3.3 Full-flag CLI (for reproducibility)

Every flag explicit, equivalent to the minimal invocation at the time of writing:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
    --T-days <N> --replan-K 2 --seed 42 \
    --N-smc 512 --K-per-chain 1000 \
    --num-mcmc 3 --max-temp-levels 30 \
    --ctrl-n-smc 2048 --ctrl-num-mcmc 16 \
    --ctrl-chees-max 512 --ctrl-n-anchors 12 \
    --ctrl-max-levels 40 \
    --output-dir <DIR>
```

3.4 Filter-side flags

flag	value	what it controls
`-N-smc`	512	outer SMC^2 posterior particles over θ (10-D filter parameters)
`-K-per-chain`	1000	inner-PF state particles per chain (latent state cloud size)
`-num-mcmc`	3	HMC moves per outer tempering level
`-max-temp-levels`	30	cap on outer tempering levels (typically 5–6 are used)
`-ot-max-weight`	0.0	OT rescue OFF (turning it on gives $\sim$12$\times$ slowdown)
`-liu-west-a`	0.0	Liu–West θ-cloud shrinkage OFF (original bootstrap path)

Table 2: Filter-side configuration. All values are bench defaults.

3.5 Controller-side flags

flag	value	what it controls
`-ctrl-n-smc`	2048	outer SMC^2 particles over θ_{ctrl} (RBF coefficients)
`-ctrl-num-mcmc`	16	ChEES-HMC moves per controller tempering level
`-ctrl-chees-max`	512	ChEES picker upper bound; candidate list = $[16, 32, 64, \ldots, 512]$
`-ctrl-n-anchors`	12	RBF basis cardinality (controller posterior dimension)
`-ctrl-max-levels`	40	cap on controller tempering levels (typically 5–6 are used)
`-ctrl-n-inner`	64	inner CRN-MC trajectories per cost evaluation
`-ctrl-target-nats`	8.0	auto-calibration target entropy (nats)
`-ctrl-sigma-prior`	1.5	prior std on the RBF coefficient vector
`-ctrl-hmc-step`	0.2	base leapfrog step size ε
`-ctrl-hmc-leap`	16	base leapfrog count (overridden adaptively by ChEES)

Table 3: Controller-side configuration. All values are bench defaults.

3.6 Sweep launcher

The full sweep was driven by the bash launcher at `version_1_5_Julia/tools/launchers/run_julia_horizon` which loops over the requested horizons, runs the bench with `-T-days <N> -seed 42 -output-dir <$SWEEP_ROOT/T<N>d_seed42>`, samples `nvidia-smi` at 1 Hz into a per-horizon `nvidia_smi.csv`, calls the canonical state-traces plotter at `compare_v15_julia_vs_python/src/plot_state_traces.jl` explicitly (the bench's internal auto-plot path resolves under the deprecated `version_2_Julia` tree and is not used), and calls the headline-metrics extractor `compare_v15_julia_vs_python/src/extract_h` to populate `results.json` per horizon and append a row to the sweep-level `horizons_results.csv`.

The exact T-days values for the published sweep:

$$T \in \{14, 28, 42, 56, 70, 84\} \quad \text{days}, \quad \text{seed} = 42.$$

4 Functional programming paradigm in Julia

4.1 Why we moved to functional

The `version_3_julia/` stack was a Python-port-style Julia implementation: mutable structs, `!`-suffix mutating methods, threaded `MersenneTwister` state, history dicts pushed onto across stride boundaries. It accumulated subtle bugs that were hard to trace, and the user explicitly rejected it ("too much for a human to understand").

`version_1_5_Julia/` is a clean re-design. The five rules:

1. **No mutable structs in the public API.** v1.5 has exactly one struct, `PlantState`, with two fields: an `SVector{3, Float64}` and an `Int`. State updates produce a NEW `PlantState`, not a mutation of the old one.
2. **No `!`-suffix mutating methods.** Every public function returns new values. `plant_step(state, Phi_t, params, dt, key)` returns `(state=new_state, obs=...)`; the caller can build history by collecting returned values.
3. **Explicit RNG keys, JAX-style.** Pass an explicit `key::UInt64` to every randomised function. Internal sub-keys are derived deterministically via `hash((key, :sub_purpose))`. Two callers with the same `(state, key)` get the same answer, period.
4. **Pure facade over the inherently imperative GPU kernel.** KernelAbstractions kernels mutate preallocated `CuArrays` by design. v1.5 wraps every kernel-touching public function in a pure facade: `gpu_log_density( target, U, grid_obs, key)` → `Vector{Float64}` allocates a fresh output, dispatches the kernel, returns. Mutation is encapsulated; callers never see the `CuArray`.
5. **I/O isolated to the bench's top-level `main()`.** JLD2 + PNG writes happen exactly once at the end of the bench. Logging at the bench level, not inside `plant_step` / `propagate` / `obs_log_weight`.

4.2 The `SMC2FC_functional` library — now the default

Status update (2026-05-08): the functional rewrite at `julia/SMC2FC_functional/` is now *the default Julia framework*. The original port has been renamed to `julia/SMC2FC_DEPRECIATED_USE_FUNCTION` (kept as a back-compat reference for the three-library comparison harness only). A bit-identical migration test is recorded in `julia/SMC2FC_functional/docs/MIGRATION_RESULT.md`.

The functional library has the same algorithmic contract as the deprecated port, but:

- Top-level functions take immutable inputs and return immutable outputs.
- Pre-allocated buffers live behind an immutable `Workspace` container.
- No `!`-mutating functions exposed publicly.
- Multiple-dispatch-driven specialisation; idiomatic Julian.
- Google-style docstrings on every file and every function.

The drop-in bench at
`julia/SMC2FC_functional/benchmarks/gpu_drop_in/bench_dropin.jl`
is byte-equivalent to the v1.5 GPU bench except for one import line (`using SMC2FC_functional: run_tempered_smc_gpu`). The three-library cross-check harness lives at `julia/SMC2FC_functional/benchmarks/compare_three_libraries.jl` and the seven verification gates are written up in `julia/SMC2FC_functional/docs/GATE_RESULTS.md`.

5 From LaTeX to Lean4 — the formal specification layer

This section documents the first half of the pipeline: LaTeX maths → Lean4 typed definitions → Lean4 native binary.

5.1 LaTeX source of truth

The canonical mathematical specification of FSA v1.5 is **§1 of this document**. It defines: state $y = (B, F, A)$, the three drift equations $(dB/dt, dF/dt, dA/dt)$ (with the Stuart–Landau slope $\mu(B, F)$ appearing as an auxiliary scalar inside the dA/dt equation), the state-dependent diffusion (Jacobi / CIR / CIR), the three direct-Gaussian observation channels, the parameter list (Table 1), and the time grid. The Lean4 modules below transcribe those equations one-to-one, in the same order, with no further interpretation step. If §1 changes, the corresponding Lean4 file changes; the maths and the typed reference live in lock-step.

5.2 Lean4 module layout

The Lean4 source tree lives at `version_1_5_LEAN/`. The modules that encode FSA v1.5 are in `Fsa/V15/`:

File (in `Fsa/V15/`)	Defines	Maps to (in §1)
`Types.lean`	`PlantState`, `ParamsV15`, `ParamsV1`	state y, the two parameter bases
`Truth.lean`	`DEFAULT_PARAMS`, `INIT_STATE`	truth values in Table 1, y_0
`Adapters.lean`	`paramsV15ToV1`	v1.5 → v1 basis rotation
`Dynamics.lean`	`drift`, `diffusion_state_dep`, `em_step_substepped`, `reflect_unit`	drift, diffusion, EM step, $[0, 1]$ reflection
`Plant.lean`	`plantStep`	one Euler–Maruyama step + obs sample
`Estimation.lean`	`obsLogWeightOne`, `applyPrior`	observation log-likelihood, prior transform

Table 4: Lean4 modules and the LaTeX they correspond to. Each module is a straight transcription, no novel mathematics. Full path is `version_1_5_LEAN/Fsa/V15/`.

The CLI entry point is `version_1_5_LEAN/Main.lean`, which imports `Fsa.V15` and dispatches JSON-on-stdin to the appropriate module function.

5.3 Build

From the project root:

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_LEAN
lake build
```

This produces the native binary at
`version_1_5_LEAN/.lake/build/bin/fsa_v15_cli`
which is what the differential-test harness in §7 talks to.

5.4 JSON dispatch protocol

`Main.lean` reads one JSON object per line on stdin, dispatches on the `"fn"` field, and writes one JSON object per line on stdout. The eight tags it accepts:

"fn" tag	Lean4 handler	Inputs → outputs
`drift`	`Dynamics.drift`	$(state, \phi, params_{v1}) \to deriv$
`diffusion`	`Dynamics.diffusion_state_dep`	$(state, params_{v1}) \to \sigma$
`emStep`	`Dynamics.em_step_substepped`	$(state, params_{v1}, noise, \phi, dt, n_{sub}) \to next$
`reflectUnit`	`Dynamics.reflect_unit`	$x \to x'$
`plantStep`	`Plant.plantStep`	$(state, \phi, params_{v15}, obs\_params, dt, sde\_n, obs\_n) \to (next, ob$
`obsLogWeight`	`Estimation.obsLogWeightOne`	$(B, F, A, obs, obs\_params) \to \log w$
`paramsV15ToV1`	`Adapters.paramsV15ToV1`	$params_{v15} \to params_{v1}$
`applyPrior`	`Estimation.applyPrior`	$(kind, \mu, \sigma, u) \to x$

Table 5: The eight JSON dispatch tags accepted by `fsa_v15_cli`. Handlers are wired in `Main.lean:166-218`; each delegates to a single function in the corresponding module.

Worked example. A single `drift` call:

```
$ echo '{"fn":"drift","state":[0.5,0.3,0.1],"phi":1.0,
  "params":{"tau_B":42.0,"tau_F":7.0,"kappa_B":0.012,"kappa_F":0.030,
   "epsilon_A":0.40,"lambda_A":1.00,"mu_0":0.02,"mu_B":0.30,
   "mu_F":0.10,"mu_FF":0.40,"eta":0.20,"sigma_B":0.010,
   "sigma_F":0.012,"sigma_A":0.020}}' | fsa_v15_cli

{"deriv":[<dB/dt>,<dF/dt>,<dA/dt>]}
```

The reply contains exactly the three-vector that the LaTeX equation (dB/dt, dF/dt, dA/dt in §1) prescribes, evaluated at the requested (B, F, A) and ϕ. No state is kept across requests; the binary is purely functional in/out and the process can be left running for the duration of the diff test (§7).

Note on parameter bases. `drift`, `diffusion`, and `emStep` accept the v1 NamedTuple (κ_B, κ_F, τ_B, τ_F, …; 14 fields). `plantStep` accepts the v1.5 form (B_∞, F_∞ in place of κ_B, κ_F). `paramsV15ToV1` is the adapter between them. This split mirrors the Julia-side convention.

6 From Lean4 to Julia — the executable port layer

This section documents the second half: Lean4 reference ↔ Julia implementation. The Julia model code lives at `version_1_5_Julia/models/fsa_high_res/` in seven files; five of those have a Lean4 twin, two do not (the GPU layer; see §8).

6.1 One-to-one function map

Each Lean4 function is exercised by the differential test (§7) against its Julia twin. The pairing:

6.2 The three high-risk transitions: `@match` sites

The diff test calls out three `@match` sites by name in its testset comments:

1. `paramsV15ToV1` (`Simulation.params_v15_to_v1_nt`) — the basis rotation. Two parameterisations of the same 14-field struct meet here. The diff test compares all 14 output fields field-by-field at 10^{-6}.
2. `reflectUnit` (inlined in `Plant.plant_step`) — the $[0, 1]$ boundary reflection on B. Tested with 10 corner cases (boundaries, just-outside, deep-outside) plus 5 random uniform draws.

Lean4 function	Julia function	Tested at tole
`Dynamics.drift`	`Dynamics.drift` (`_dynamics.jl`)	10^{-6}
`Dynamics.diffusion_state_dep`	`Dynamics.diffusion_state_dep` (`_dynamics.jl`)	10^{-6}
`Dynamics.em_step_substepped`	`Dynamics.em_step_substepped` (`_dynamics.jl`)	10^{-4}
`Dynamics.reflect_unit`	inlined boundary check (`_plant.jl`)	10^{-6}
`Plant.plantStep`	`Plant.plant_step` (`_plant.jl`)	10^{-6}
`Estimation.obsLogWeightOne`	`Estimation.obs_log_weight` (per-particle slice, `estimation.jl`)	10^{-6}
`Adapters.paramsV15ToV1`	`Simulation.params_v15_to_v1_nt` (`simulation.jl`)	10^{-6}
`Estimation.applyPrior`	`apply_prior` (`gpu_pf.jl`, `estimation.jl`)	10^{-6}

Table 6: Lean4 ↔ Julia function map. The 10^{-4}-tolerance row is the only one that integrates noise (4-substep Euler–Maruyama); the rest are single-step deterministic operations. File:line references are tracked in the diff-test source (`version_1_5_Julia/diff_test/test_lean_diff_v15.jl`).

3. `applyPrior` (`apply_prior`) — the LogNormal / Normal prior transform that maps unconstrained u to the constrained parameter. Tested with 7 LogNormal corner cases (including the ± 25 clamp boundaries) and 5 Normal random draws.

These three are the highest-risk transitions because each is a small piece of code that "looks obvious" but encodes a non-obvious mathematical convention (basis change, reflection, parameter constraint). The diff test names them explicitly so that any future agent who breaks one is caught immediately.

6.3 Functions in Julia with no Lean4 twin (CPU side)

These exist in the Julia model files but are not separately diff-tested; they are array-level wrappers whose correctness inherits from the per-particle primitives above:

- `Plant.plant_rollout` (`_plant.jl`) — multi-bin rollout, repeatedly applies `plant_step`. Correctness follows from `plant_step` being correct.
- `Estimation.propagate` (`estimation.jl`) — advance an M-particle cloud one EM step. Correctness follows from the per-particle drift / diffusion / reflect being correct.
- `Estimation.PARAM_NAMES`, `Estimation.PARAM_PRIOR_CONFIG` — configuration constants, not functions.

The GPU files `gpu_pf.jl` and `gpu_control.jl` also have no Lean4 twin; §8 treats them as a separate verification gap because their content is qualitatively different (whole new algorithms, not just array-level wrappers).

7 The differential test harness — verifying parity

The harness lives at
`version_1_5_Julia/diff_test/test_lean_diff_v15.jl`
and is the executable proof that the Julia and Lean4 sides agree. This section walks through it end-to-end so a future reader can re-run it, modify it, and trust its outputs.

7.1 What it does, in one paragraph

For each of the eight Lean4-exposed functions in Table 5, the harness generates inputs (random states, random parameters, pre-drawn noise), sends one JSON request to the `fsa_v15_cli`

subprocess, runs the same call locally in Julia, and asserts `|Julia - Lean| < tolerance`. Five random trials per function plus corner cases for the boundary-reflection and prior-clamp tests. The whole run takes a few seconds.

7.2 Subprocess setup

The Lean4 binary is opened once at start of test and left running for the duration:

```
function open_lean_client()
    isfile(LEAN_BIN) || error("LEAN binary not found at $LEAN_BIN ...")
    io = open(`$LEAN_BIN`, "r+")
    return LeanClient(io)
end
```

(`test_lean_diff_v15.jl:42-63`). One process, one stdin, one stdout, line-buffered. The `round_trip` helper writes one JSON line, `flush`es, reads one line, parses it. JSON values allow $\pm$`Inf` via `allow_inf=true` on both encode and decode.

7.3 Pre-drawn noise convention

For `emStep` and `plantStep` the harness pre-draws all noise on the Julia side and passes it as an array to *both* the Julia call and the Lean4 call:

```
function _random_noise(rng)
    randn(rng, 3)
end
```

(`test_lean_diff_v15.jl:107-109`). This eliminates RNG-divergence as a noise source — the test is purely about the deterministic numerical agreement of the math, not about whether two languages happen to produce the same Mersenne-Twister stream from the same seed (they don't).

7.4 Tolerance ladder

Two tolerances are used, set at the top of the file (`test_lean_diff_v15.jl:36-37`):

- `SINGLE_STEP_TOL = 1e-6` for: `drift`, `diffusion`, `reflectUnit`, `paramsV15ToV1`, `plantStep`, `obsLogWeight`, `applyPrior`. These are deterministic single-step operations; agreement at 10^{-6} is the IEEE-754 round-off floor for the relevant compositions.
- `INTEGRATED_TOL = 1e-4` for: `emStep` (4-substep Euler–Maruyama). The looser tolerance accounts for accumulated floating-point error across substeps and one Wiener increment.

A failure at either tolerance is by construction a Julia bug (or, in principle, a Lean4 bug — but in practice the Lean4 side type-checks and is much shorter, so the asymmetry is stable).

7.5 Per-testset coverage

7.6 Run command

To reproduce the verification end-to-end:

```
# 1. Build the Lean4 binary (only needed once per Lean4 source change):
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_LEAN
lake build

# 2. Run the diff test:
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
julia --project=. diff_test/test_lean_diff_v15.jl
```

Testset	Inputs per session	Tolerance
`reflect_unit`	10 corner + 5 random	10^{-6}
`apply_prior`	7 LogNormal + 5 Normal	10^{-6}
`params_v15_to_v1`	1 (14 fields each compared)	10^{-6}
`drift`	5 random	10^{-6}
`diffusion_state_dep`	5 random	10^{-6}
`em_step_substepped`	5 random with noise	10^{-4}
`plant_step`	5 random with sde + obs noise	10^{-6}
`obs_log_weight_one`	5 random	10^{-6}

Table 7: Coverage of the eight testsets in `test_lean_diff_v15.jl`. "Random" inputs are drawn uniformly from the physiological-bound box $B \in [0, 1]$, $F \in [0, 1]$, $A \in [0, 1]$.

The expected output is a green Julia testset summary with no `Error`s and no `Fail`s. If any testset goes red, the relevant (input, Julia output, Lean4 output) triple is printed, and the diff is by construction a Julia (or Lean4) bug at that function.

7.7 Pipeline flow diagram

Figure 1 shows the data-flow from LaTeX maths to a green diff test, including which files are involved at each stage.

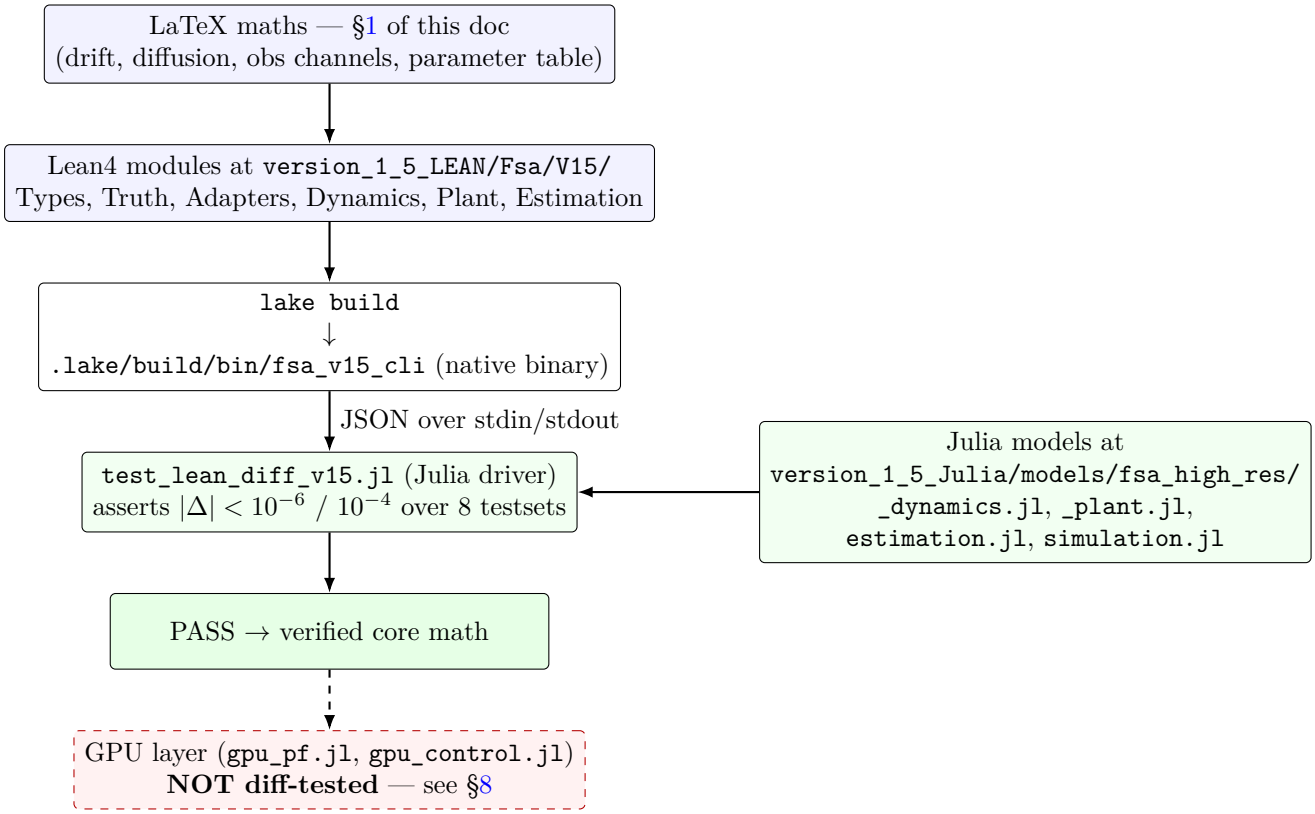

Figure 1: LaTeX → Lean4 → Julia pipeline for FSA v1.5. Solid arrows are tested data-flow; the dashed arrow into the GPU layer marks an unverified handoff (per §8). Every box names the on-disk file path needed to reproduce that stage.

8 Verification gap — what the diff test does NOT cover

The differential test in §7 verifies the eight core SDE primitives at $10^{-6}/10^{-4}$ tolerance. Two GPU files in the same model directory are *not* covered:

- `version_1_5_Julia/models/fsa_high_res/gpu_pf.jl`
- `version_1_5_Julia/models/fsa_high_res/gpu_control.jl`

This section records what is in those files and why it isn't currently diff-tested.

8.1 Concrete gap inventory

Layer	File:line (approx.)	What it does	
Control cost rollout	`gpu_control.jl`, `fsa_v1_cost_kernel!` (lines 54–147)	RBF-parameterised $\Phi(t)$, substepped EM forward simulation, soft fatigue penalty per Eq. 9	
Segmented PF kernel	`gpu_pf.jl`, `propagate_segment_kernel!` (lines 53–143)	per-particle SDE + obs log-weight in a multi-bin loop, NaN guard on out-of-domain particles	inner SDE math duplicat
HMC leapfrog	`gpu_pf.jl`, `parallel_hmc_one_move` (lines 555–610)	per-chain leapfrog with prior gradient	
FD gradient batcher	`gpu_pf.jl`, `gpu_grads` (lines 506–543)	central-difference gradient on GPU	
NaN guard fallback	`gpu_pf.jl`, line 120 (approx.)	resets a particle to the init state if a step produces NaN	
OT rescue / ESS	`gpu_pf.jl`, lines 270–298 (approx.)	resampling-rescue heuristics under near-degenerate weights	

Table 8: What's in the GPU layer that the differential test doesn't exercise. Line ranges are approximate (rounded to the nearest 10); re-grep at audit time if the files have moved.

8.2 Two qualifying notes

1. The per-particle SDE math *inside* the GPU kernels duplicates the verified core line-by-line. The drift, diffusion, $[0, 1]$ reflection, and 3-channel Gaussian obs log-weight that `propagate_segment_kernel!` executes are textually equivalent to the corresponding code in `_dynamics.jl` / `_plant.jl` / `estimation.jl` (which is what the diff test verifies). So the GPU kernels could in principle be diff-tested by adding a single-thread debug path that re-uses the same JSON CLI. Today they aren't — this is a future-work item, not a known bug.

2. What the GPU code is currently trusted on. End-to-end empirical parity with the Python+JAX reference, written up in `compare_v15_julia_vs_python/docs/julia_vs_python_v15_writeup.tex` (see [1]). That gives a *statistical* guarantee at the closed-loop bench level, not a per-function guarantee at 10^{-6}. The two are complementary: the diff test pins the SDE primitives; the empirical comparison pins the full pipeline including the GPU algorithms.

Status. Marked as future work. Not in scope of this documentation pass. The recommended next step — if anyone wants to close this gap — is to expose a single-thread, single-particle debug path in each GPU kernel that emits the same JSON the diff test already understands, and add new testsets to `test_lean_diff_v15.jl` for them.

References

[1] *FSA v1.5 closed-loop SMC²-MPC: a side-by-side Python+JAX vs Julia comparison study.* 2026-05-08. compare_v15_julia_vs_python/docs/julia_vs_python_v15_writeup.pdf.

[2] *Current best Julia v1.5 SMC²-FC config (RTX 5090, T=28d).* 2026-05-08. compare_v15_julia_vs_python/docs/technical_guide_to_current_best_Julia_SMC2FC_config.pdf.